Лексическое окружение и контекст

- Введение в контекст
- Контекст и ключевое слово this
- Явная привязка контекста (call, bind, apply)
- Контекст и модули

Введение в контекст

Кроме **области видимости** (область) и **лексического окружения** (объект) в JavaScript имеется ещё **контекст**.

Контекст – это то, на что указывает ключевое слово **this**. По сути **this** – ссылка на объект, которому **принадлежит** выполняемый в данный момент код. Ключевое свойство **this** – это свойство объекта лексического окружения (Environment Record).

Поведение ключевого слова **this** зависит от контекста, в котором оно используется, и от того, в каком режиме оно используется - **строгом** или **нестрогом**.

> ***Примечание***. Контекст – критически важное понятие для освоения урока **Замыкания**. В свою очередь замыкание – один из важных инструментов веб-разработчика.

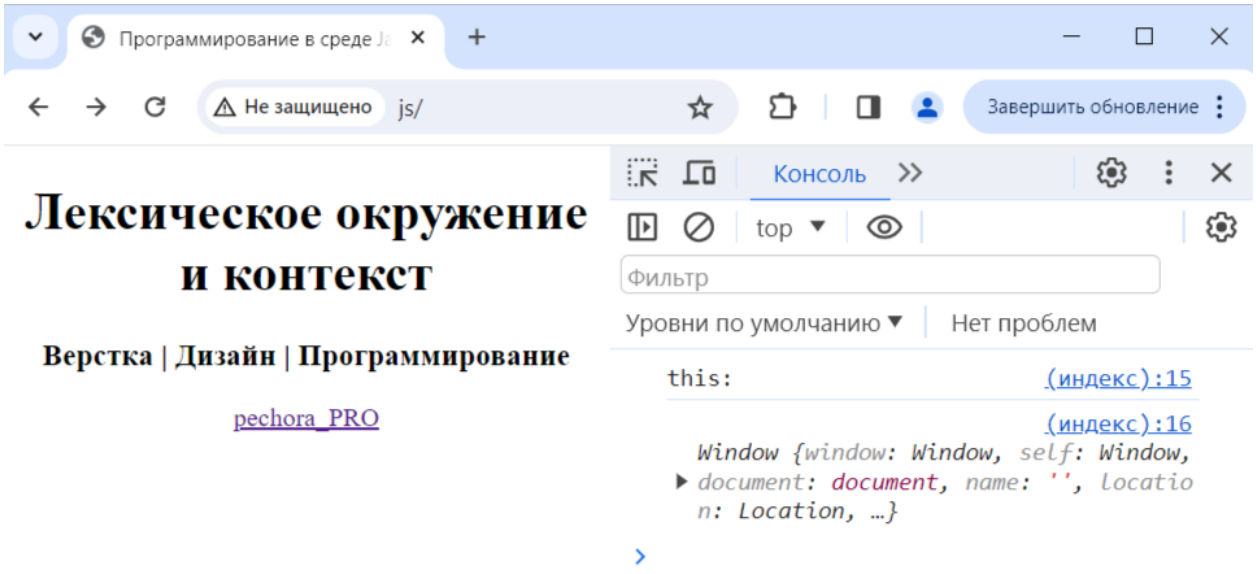
Глобальный контекст

В глобальном контексте **this** ссылается на **глобальный объект**. Глобальным объектом веб-браузера является объект **Window**.

Важно. **Window** – **глобальный объект**, который представляет окно браузера.

Консольный вывод глобального объекта можно получить при выполнении следующего кода:

```
// в глобальном контексте this ссылается на глобальный объект window
console.log('this:');
console.log(this);
```



Когда мы создаем **глобальные переменные**, то они автоматически начинают **принадлежать глобальному объекту Window** – самому главному объекту в браузере.

Например, стандартные методы, которыми мы часто пользуемся, находятся внутри объекта **Window** и являются его методами:

- console.log(),
- alert,
- prompt и т.д.

Это легко проверить. Достаточно вызвать эти методы следующим образом:

- **window**.console.log('Привет');
- **window**.alert('Привет');
- **window**.prompt('Как тебя зовут?');

Ссылка **this** – это не всегда глобальный объект. Во вложенном (локальном) лексическом окружении, **this** это ссылка на **родительское лексическое окружение**. Или, другими словами, ссылка на **объект**, которому принадлежат переменные и функции текущей области видимости.

Разобраться со ссылкой **this** и ее поведением во вложенных областях и есть задача урока.

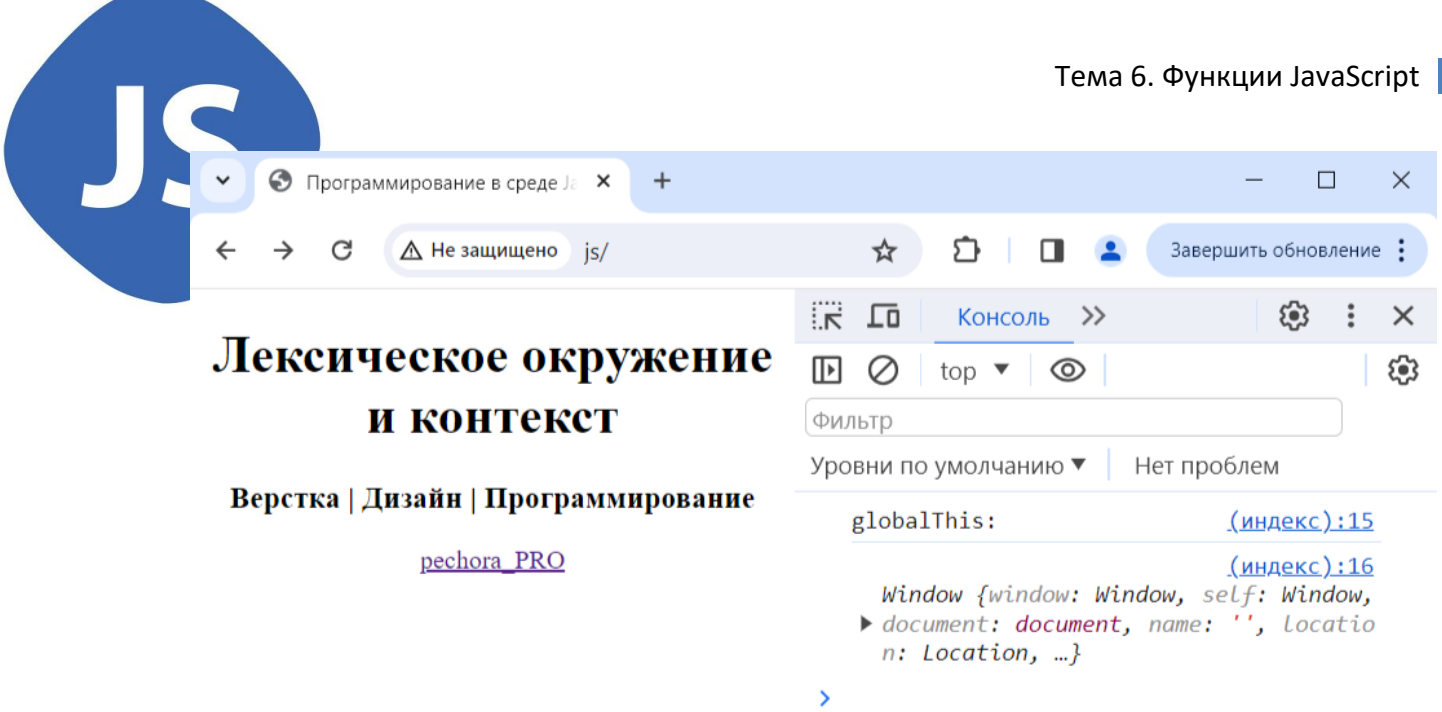
Глобальный контекст globalThis

Так как в локальном контексте **this** не всегда **глобальный объект Window**, в стандарт **ES2020** было добавлено определение объекта **globalThis**.

Объект **globalThis** позволяет ссылаться на **глобальный контекст** вне зависимости, в какой среде и в какой ситуации выполняется код:

```
// globalThis всегда глобальный объект
console.log('globalThis:');
console.log(globalThis);
```

Варианты поведения локального **this** в зависимости от среды и внешних условий разберем в следующих демонстрационных примерах.



В демонстрационном примере **example_1** приведен набор переменных и функций, принадлежащих глобальному контексту.

example_1.

```
<script>

    // набор глобальных переменных

    var aVar = "I am a VAR variable";

    let aLet = "I am a LET variable";

    const aCONST = 'I am a CONST';

    aWithOut = "I am a WITHOUT variable";

    // функция инструкция

    function aFnStatement() {

        console.log('I am a FUNCTION STATEMENT');

    }

    // функция выражение

    var aFnExpression = function () {

        console.log("I'm a FUNCTION EXPRESSION");

    }
```

```
    // функция стрелка
    var aFnArrow = () => {
        console.log("I'm a FUNCTION ARROW");
    }
    // объект
    var aMyObj = {
        name: "master",
        login: "qwerty",
        fn: function() {
            console.log("I am a FUNCTION METHOD");
        }
    };
    console.log(this.aVar); // I am a VAR variable
    console.log(this.aLet); // undefined
    console.log(this.aCONST); // undefined
    console.log(this.aWithOut); // I am a WITHOUT variable
    console.log(this.aFnStatement); // function aFnStatement()
    console.log(this.aFnExpression); // function aFnExpression()
    console.log(this.aFnArrow); // function aFnArrow()
    console.log(this.aMyObj); // Object
</script>
```

Теперь более подробно.

Контекст и ключевое слово this

Ссылка **this** определяет объект, которому принадлежит выполняемый в данный момент код.

1. В контексте глобальной области видимости **this** – это **глобальный объект Window**.
2. В контексте функции, **this** зависит от того, как **вызывается** функция:
   - если функция определена как функция-инструкция или функция-выражение, то **this** в не строгом режиме указывает на **глобальный объект**, а в строгом – **undefined**;
   - если функция определена как функция-стрелка, тогда **this** – это **глобальный объект**;
   - если функция определена внутри объекта (как метод объекта), тогда **this** – это **объект**, который использовался для вызова метода. Если метод – стрелка, **this** – **глобальный объект**.

Теперь тоже самое, но в примерах демонстрационного кода.

Контекст глобальной области видимости

В контексте глобальной области видимости (вне модулей и функций) **this** – это глобальный объект.

В демонстрационном примере **example_2** определяем глобальные функции и объект, принадлежащие глобальному объекту **this**. В свою очередь, каждая из объявленных структур имеет свой контекст **this**.

example_2.

```
<script>

    // функция инструкция

    function aFnStatement() {

        console.log('I am a FUNCTION STATEMENT');
```

```
    console.log(this);
}
// функция выражение
var aFnExpression = function () {
    console.log("I'm a FUNCTION EXPRESSION");
    console.log(this);
}
// функция стрелка
var aFnArrow =  () => {
    console.log("I'm a FUNCTION ARROW");
    console.log(this);
}
// объект
var aMyObj = {
    methodExpr: function() {
        console.log("I'm a METHOD Expression Object");
        console.log(this);
    },
    methodArrow: () => {
        console.log("I'm a METHOD Arrow Object");
        console.log(this);
    }
};
// глобальный объект window
console.log(this);
aFnStatement(); // window
```

```
    aFnExpression(); // window
    aFnArrow(); // window
    aMyObj.methodExpr(); // Object
    aMyObj.methodArrow(); // window
</script>
```

Важно. В следующих примерах обращайте внимание не только на контекст **this**, но и на **всплытие** локальных переменных уровня блока. Обратите внимание, ссылка на контекст и ссылка на локальную переменную очень разные ссылки (будьте внимательны).

Контекст функции инструкции

В пределах функции инструкции ссылки **this** и **globalThis** ссылаются на внешний контекст. Для функций, определенных в **глобальном контексте** — это объект **Window**.

example_3.

```
<script>
    // функция инструкция
    function fnStatement() {
        // локальная переменная name
        let name = "I'm Silvija";
        console.log(name); // I'm Silvija
        console.log(this.name); // My name is Bond. James Bond
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
```

```
    fnStatement();
</script>
```

В демонстрационном примере **example_4** рассмотрим контекст **вложенных** функций.

example_4.

```
<script>
    // внешняя функция инструкция
    function fnOuterStatement() {
        // локальная переменная name
        let name = "I'm Silvija";
        // вложенная функция инструкция
        function fnInnerStatement() {
            // локальная переменная name
            let name = 'Who am I?';
            console.log(name); // Who am I?
            console.log(this.name); // My name is Bond. James
            Bond
        }
        // вызов вложенной функции
        fnInnerStatement();
     };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов внешней функции
    fnOuterStatement();
</script>
```

Если использовать строгий режим - **strict mode**, то в этом случае **this** имеет значение **undefined**. **GlobalThis** значение не меняет.

example_5.

```
<script>
    // внешняя функция инструкция
    function fnOuterStatement() {
        // включаем строгий режим
        "use strict";
        // локальная переменная name
        let name = "I'm Silvija";
        // вложенная функция инструкция
        function fnInnerStatement() {
            // локальная переменная name
            let name = 'Who am I?';
            console.log(name); // Who am I?
            console.log(this); // undefined
            console.log(globalThis); // window
        }
        // вызов вложенной функции
        fnInnerStatement();
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов внешней функции
    fnOuterStatement();
</script>
```

Таким образом. Контекст (this) – функции **инструкции** всегда глобальный **window**. В режиме **strict mode** – **undefined**.

Контекст функции выражения

Поведение функций выражений похоже на поведение функций инструкций.

example_6.

```
<script>
    // внешняя функция выражение
    fnOuterExpression = function () {
        // локальная переменная name
        let name = "I'm Silvija";
        // вложенная функция выражение
        fnInnerExpression = function() {
            // локальная переменная name
            var name = "Who am I?";
            console.log(name); // Who am I?
            console.log(this.name); // My name is Bond. James
            Bond
        }
        // вызов вложенной функции
        fnInnerExpression();
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
```

```
    // вызов внешней функции
    fnOuterExpression();
</script>
```

Если использовать строгий режим - **strict mode**, то **this** имеет значение **undefined**. **GlobalThis** значение не меняет.

example_7.

```
<script>
    // внешняя функция выражение
    fnOuterExpression = function () {
        // включаем строгий режим
        'use strict';
        // локальная переменная name
        var name = "I'm Silvija";
        // вложенная функция выражение
        let fnInnerExpression = function() {
            console.log(name); // I'm Silvija
            console.log(this); // undefined
            console.log(globalThis); // window
        }
        // вызов вложенной функции
        fnInnerExpression();
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов внешней функции
```

```
    fnOuterExpression();
</script>
```

Таким образом. Контекст (this) – функции **выражения** всегда глобальный **window**. В режиме **strict mode** – **undefined**.

Контекст стрелочных функций имеет свои особенности (они и правда, какие-то странные ;).

Контекст стрелочной функции

Поведение стрелочных функций на первых порах может преподносить некоторые сюрпризы.

Рассмотрим примеры.

example_8.

```
<script>
    // внешняя функция стрелка
    fnOuterArrow = () => {
        // локальная переменная name
        let name = "I'm Silvija";
        // вложенная функция стрелка
        let fnInnerArrow = () => {
            console.log(name); // I'm Silvija
            console.log(this.name); // My name is Bond. James
            Bond
        }
        // вызов вложенной функции
```

```
        fnInnerArrow();
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов внешней функции
    fnOuterArrow();
</script>
```

В строгом режиме стрелочная функция контекст не меняет.

example_9.

```
<script>
    // внешняя функция стрелка
    fnOuterArrow = () => {
        // включаем строгий режим
        'use strict';
        // локальная переменная name
        let name = "I'm Silvija";
        // вложенная функция стрелка
        let fnInnerArrow = () => {
            console.log(name); // I'm Silvija
            console.log(this.name); // My name is Bond. James
            Bond
        }
        // вызов вложенной функции
        fnInnerArrow();
    };
```

```
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов внешней функции
    fnOuterArrow();
</script>
```

Таким образом. Контекст (this) – функции **стрелки** всегда глобальный **window**. В режиме **strict mode** – стрелочная функция контекст **не меняет**.

Важно.

- Контекст (this) – функции **инструкции** и функции **выражение** – глобальный **window**. В режиме **strict mode** – **undefined**.
- Контекст стрелочной функции всегда **window**.

Примечание. Не путайте контекст функции и всплытие переменной.

Контекст объекта

В контексте объекта, в том числе в его **методах**, ключевое слово **this** ссылается на этот же объект.

example_10.

```
<script>
    // определене объекта
    let obj = {
        mtdStatement: function () {
```

```
        console.log('Метод-выражение');
        console.dir(this);
        },
        mtdArrow: () => {
            console.log('Метод-стрелка');
            console.dir(this);
        }
    };
    // вызов методов объекта
    obj.mtdStatement();  // Object
    obj.mtdArrow();  // window
</script>
```

example_11.

```
<script>
    myFn = function () {
        // локальная переменная name
        var name = "I'm Silvija";
        console.log(name);  // I'm Silvija
        console.log(this.name);  // No Name
        console.log(globalThis.name); // My name is Bond. James
        Bond
    }
    // определене объекта
    let obj = {
        // name - свойство объекта
```

```
        name: "No Name",
        mtdStatement: myFn,
        mtdArrow: () => {
            // локальная переменная name
            console.log(name);  // My name is Bond. James Bond
            console.log(this.name);  // My name is Bond. James
            Bond
            console.log(globalThis.name); // My name is Bond.
            James Bond
        }
    };
    // глобальная переменная name
    var name = "My name is Bond. James Bond";
    // вызов методов объекта
    obj.mtdStatement();
    // obj.mtdArrow();
</script>
```

Важно.

- Контекст метода (функции) **выражения** ссылается на сам **объект**.
- Контекст метода (функции) **стрелки** – глобальный **window**.



Явная привязка контекста (call, bind, apply)

В JavaScript при вызове функции можно установить нужный **this**, т.е. контекст, в котором она должна выполняться. С помощью методов **call**() и **apply**() можно задать **явную привязку функции к определенному контексту**.

call

Метод **call**() вызывает функцию с указанным значением **this** и индивидуально предоставленными аргументами.

```
myFunc.call(thisArg, arg1, arg2, ..., argN);
```

Описание

В представленной функции **myFunc**, первый аргумент метода **call**() **–** thisArg становится её контекстом – **this**. После контекста передаются аргументы для функции **myFinc**, аргументы передаются как есть.

Параметры

- **thisArg** – значение, которое будет использоваться, как **this** при вызове функции **myFunc**.
- **arg1**, …, **argN** – необязательный. Аргументы функции.

Возвращаемое значение

Результат вызова функции с указанным **this** значением и аргументами.

Обратите внимание, вызов:

```
func.call (context, a, b, c);
```

– **эквивалентен** обычному вызову функции:

```
func (a, b, c);
```



но с явно указанным контекстом **this** (context).

example_12.

```
<script>
    // глобальные переменые
    var login = "Login is not defined";
    var pwd = "Password is not defined";
    var email = "Email is not defined";
    // объект user
    var user = {
        // свойства объекта
        login: "Den",
        pwd: "SuperAdmin",
        email: "denis@gmail.ru"
    }
    // определение функции вывода привязанных к контексту данных
    function fnOutData()  {
        console.log(this.login);
        console.log(this.pwd);
        console.log(this.email);
    }
    // вызов функции в контексте глобального объекта
    window.fnOutData();
    // вызов функции в контексте глобального объекта
    // альтернативный вариант
    // window.fnOutData.call(this);
    // вызоа функции в контексте объекта 'user'
```

```
    // window.fnOutData.call(user);
</script>
```

Любая функция может в своём коде использовать контекст **this**, каким будет это значение – выяснится **в момент запуска** функции.

example_13.

```
<script>
    // дефолтные переменные глобального window
    var id = 13;
    var name = "LADA Kalina";
    var brand = "АвтоВАЗ";
    var type = "Бензиновый";
    var power = "90";
    var image = "lada-kalina.jpg";
    // переменные можно сохранить в объекте
    var lada = {
        id: 13,
        name: "LADA Kalina",
        brand: "АвтоВАЗ",
        type: "Бензиновый",
        power: "90",
        image: "lada-kalina.jpg"
    }
    // объект toyota
    const toyota = {
        id: 2,
```

```
        name: "Toyota Land Cruiser",
        brand: "Toyota",
        type: "Дизельный",
        power: "249",
        image: "land-cruiser.jpg"
    }
    // объект kia
    const kia = {
        id: 7,
        name: "Kia Sorento Prime",
        brand: "Kia",
        type: "Дизельный",
        power: "200",
        image: "kia-sorento-prime.png"
    }
    // объявление функции outData
    const fnOutData = function(msg) {
        // контекст this будет определен во время вызова
        let html = `
            <h2>${msg}</h2>
            <h3>${this.name}</h3>
            Бренд: <b> ${this.brand} </b><br>
            Тип двигателя: <b> ${this.type} </b><br>
            Мощность (л.с): <b> ${this.power} </b><br>
            <img src="images/${this.image}" width='250px'> <br>
        `;
```

```
        return html;
    }
    // в каждый из вызовов добавим дополнительный параметр
    // функция в контексте объекта
    document.write(fnOutData.call(kia, "Корейский автопром рулит?"));
    // функция в контексте объекта
    // document.write(fnOutData.call(toyota, "Япония, вперед!"));
    // функция в контексте глобального window
    // document.write(fnOutData.call(this, "Упс... Это вам )"));
    // функция в контексте объекта глобального window
    // document.write(fnOutData.call(this.lada, "Можно оформить объектом... )"));
</script>
```

example_14.

```
<script>
    let fnDeclare = {
        // объявление метода объекта
        mtdOutData: function(msg) {
            // контекст this будет определен во время вызова
            let html = `
                <h2>${msg}</h2>
                <h3>${this.name}</h3>
                Бренд: <b> ${this.brand} </b><br>
                Тип двигателя: <b> ${this.type} </b><br>
```

```
            Мощность (л.с): <b> ${this.power} </b><br>
            <img src="images/${this.image}" width='250px'>
            <br>
        `;
        return html;
    }
}
// объект lada
let lada = {
    id: 13,
    name: "LADA Kalina",
    brand: "АвтоВАЗ",
    type: "Бензиновый",
    power: "90",
    image: "lada-kalina.jpg"
}
// объект toyota
let toyota = {
    id: 2,
    name: "Toyota Land Cruiser",
    brand: "Toyota",
    type: "Дизельный",
    power: "249",
    image: "land-cruiser.jpg"
}
// объект kia
```

```
    let kia = {
        id: 7,
        name: "Kia Sorento Prime",
        brand: "Kia",
        type: "Дизельный",
        power: "200",
        image: "kia-sorento-prime.png"
    }
    // в каждый из вызовов добавим дополнительный параметр
    // функция в контексте объекта
    document.write(fnDeclare.mtdOutData.call(kia, "Корейский автопром рулит?"));
    // функция в контексте объекта
    // document.write(fnDeclare.mtdOutData.call(toyota, "Япония, вперед!"));
    // функция в контексте глобального window
    // document.write(fnDeclare.mtdOutData.call(lada, "Владейте на здоровье :("));
</script>
```

Если заранее неизвестно, с каким количеством аргументов понадобится вызвать функцию, можно использовать более мощный метод: **apply**().

apply

Метод **apply**() аналогичен методу **call**(). Единственное отличие в том, что аргументы в **apply** передаются в виде массива.

```
myFunc.apply(thisArg, argsArray);
```

Описание

Если в **call**() параметры функции передаются через запятую, то метод **apply**() должен передавать параметры функции в массиве.

Параметры

- **thisArg** – Значение **this**, предоставленное для вызова функции **myFunc**.
- **argsArray** – Необязательный. Объект, определяющий аргументы, с которыми следует вызывать функцию, или **null** если функции не следует предоставлять аргументы.

Возвращаемое значение

Результат вызова функции с указанным **this** значением и аргументами.

Обратите внимание, вызов:

```
func.apply (context, array);
```

– **эквивалентен** обычному вызову функции:

```
func (...rest);
```

но с явно указанным контекстом **this** (context).

example_15.

```
<script>
    // массив для передачи в функцию
    let block = Array(
        `<header>
            <h1 align="center">Лексическое окружение и
            контекст</h1>
        </header>`,
        `<footer align="center">
            <h3>Верстка | Дизайн | Программирование</h3>
            <a href="https://vk.com/pechora_pro"
```

```
        target="_blank">pechora_PRO</a>
    </footer>`
);
/*
    block[0] - header
    block[1] - footer
*/
// набор глобальных переменных
var name = "No name";
var login = "undefined";
var email = "hidden";
var role = "lamer";
var image = "no.jpg";
// объект
var user = {
    name: "Злой Бармалей",
    login: "SuperAdmin",
    email: "super-admin@mail.org",
    role: "administrator",
    image: "user.jpg"
}
// параметр - массив
function outUserInfo(...rest) {
    // формирование блока для вывода
    let html = `
        ${rest[0]}
```

```
            ${this.name} <br>
            ${this.login} <br>
            ${this.email} <br>
            <img src="${this.image}" width="250px"> <br>
            ${rest[1]}
        `;
        // возвращаем блок вывода
        return html;
    }
    // параметр - массив block
    document.write(outUserInfo.apply(this, block));
    // document.write(outUserInfo.apply(user, block));
</script>
```

example_16.

```
<script src="orders.js"></script>
<!-- /// -->
<script>
    // номер счета
    var bill = "N-1103/12-1";
    // функция поиска заказа по номеру счета
    orderSearch = function () {
        let obj = orders.find ( obj => {
            return obj.bill == bill;
        });
        // возвращаем объект-заказ
```

```
    return {
        vendor: obj.vendor,
        product: obj.product
    };
}
// функция вывода заказа
function outProduct(name, date, payment) {
    document.write (`
        <h3>${this.title}</h3>
        Заказчик: <b>${name}</b> <br>
        CPU / ${this.cpu} <br>
        GPU / ${this.gpu} <br>
        <img src='images/${this.img}'> <br>
        Счет: от ${date}. Оплачено: ${payment}
        <hr>
    `);
}
// поиск счета
let order = orderSearch();
/*
    структура возвращаемого объекта
    order = {}
    order.vendor
    order.product
*/
// для каждого товара счета вызываем outProduct для вывода
```

```
    for (product of order.product){
        // функцию вызываем в контексте объекта product
        outProduct.apply(product, [order.vendor.name,
        order.vendor.date, order.vendor.payment]);
    }
</script>
```

Фрагмент файла данных **orders.js** представлен в листинге **example_16/orders.js**.

example_16/orders.js.

```
var orders = [
{
    bill: "N-2201/11-5",
    vendor: {
        date: "15.01.2023",
        name: "Иоганн Кристоф Фридрих фон Шиллер",
        discount: 0,
        payment: "Нет"
    },
    product: [
        {
            id: 2,
            title: 'Ультрабук HUAWEI MateBook D 15 BoM-WFQ9',
            gpu: 'AMD Radeon Graphics',
            cpu: 'AMD Ryzen 5 5500U',
            cpu_cores: 6,
            ram: '16 ГБ',
```

```
            diagonal: "15.6'",
            price: 62.999,
            img: "huawei-matebook-d-15bom-wfq9.jpg"
        }
    ]
},
// ...
]
```

bind

Метод **bind**() создает **новую функцию**, с указанным значением **this** и индивидуально предоставленными аргументами.

```
fun = myFunc.bind(thisArg, argsArray);
```

Описание

Функция **bind**() создает новую связанную функцию. Вызов связанной функции обычно приводит к выполнению функции, которую она обертывает, которая также называется целевой функцией. Связанная функция сохранит переданные параметры, включая значение **this** как свое внутреннее состояние.

Возвращаемое значение

Копия данной функции с указанным **this** значением и начальными аргументами (если они предусмотрены).

Обратите внимание, вызов:

```
fn = func.bind (context, a, b, c);
```

– **эквивалентен** обычному вызову функции:

```
fn = func (a, b, c);
```

но с явно указанным контекстом **this** (context).

example_17.

```
<script>
    // набор глобальных переменных
    var title = "Not defined";
    var brand = "Not defined";
    var color = "Not defined";
    var country = "Not defined";
    var img = "no-image.jpg";
    // массив объектов
    var clothes = [
    {
        id: 1,
        title: 'Утепленная стеганая куртка',
        brand: 'MARCO DI RADI',
        color: 'Синий',
        country: 'Китай',
        img: 'steganaya-marco.jpg'
    },
    {
        id: 2,
        title: 'Пальто',
        brand: 'Allsaints',
        color: 'Коричневый',
        country: 'Китай',
        img: 'palto-allsaints.jpg'
```

```
    }
];
// объект с функцией-методом для вывода
var order = {
    outOrder:   function ()  {
        document.write (`
            Название: ${this.title} <br>
            Бренд: ${this.brand} <br>
            Цвет: ${this.color} <br>
            Страна: ${this.country} <br>
            <img src='images/${this.img}' width='200px'>
            <hr>
        `);
    }
}
// метод call сравним с методом bind
order.outOrder.call(clothes[0]);
// метод bind в контексте this
fn1 = order.outOrder.bind();
// метод bind в контексте объекта
fn2 = this.order.outOrder.bind(clothes[1]);
// вызов результата
fn1();
// fn2();
</script>
```



Контекст и модули

При работе с модулями обратите внимание:

Каждый модуль имеет свою собственную область видимости.

Таким образом:

- переменные и функции, объявленные в модуле, не видны в других скриптах.

Модули всегда выполняются в отложенном (deferred) режиме точно так же, как скрипты с атрибутом **defer**. Это верно и для внешних и встроенных скриптов-модулей.

Таким образом:

- загрузка внешних модулей не блокирует обработку HTML;
- модули, даже если загрузились быстро, ожидают полной загрузки HTML документа, и только затем выполняются;
- сохраняется относительный порядок скриптов: скрипты, которые идут раньше в документе, выполняются раньше;
- модули всегда видят полностью загруженную HTML-страницу, включая элементы под ними.

При необходимости передачи данных между модулями можно воспользоваться глобальным объектом **window** (что допустимо, но не очень желательно). Или воспользоваться объектом веб-хранилища, например: **localStorage**.

При работе с объектами хранилища помните, что ключ и значение объекта должны быть **строками**. Если мы используем любой другой тип, например число, массив или объект, то он **автоматически преобразуется в строку**.

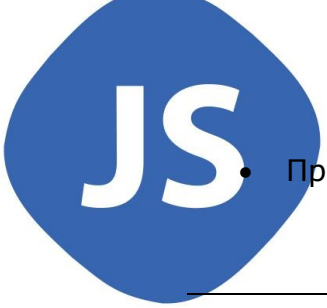
В демонстрационном примере **example_18** передадим экспортируемый из модуля массив объектов в другой скрипт (также объявленный с **type="module"**).

example_18.

```
<!-- подключение первого модуля -->
<script type="module">
    // импорт массива orders
    import { orders } from './orders.js';
    // представление массива в формат JSON
    let JSONorders = JSON.stringify(orders);
    // console.log(JSONorders);
    // сохранение представления JSON в объект хранилища
    window.localStorage.setItem('orders', JSONorders);
</script>
<!-- подключение второго модуля -->
<script type="module">
    // достаем JSON из хранилища данных
    let orders = window.localStorage.getItem('orders');
    // парсим JSON-строку в массив объектов
    orders = JSON.parse(orders);
    console.log(orders[0]); // Object
</script>
```

Для закрепления пройденного материала в **самостоятельных работах** выполним следующие задания:

- Определение ссылки this без запуска предлагаемого скрипта.
- Создание функции в определении контекста методов call, bind, apply.
- Использование привязки контекста в циклических конструкциях.
- Использование модулей и экспортируемой переменной при организации вывода данных в браузер.



- Преобразование отдельных функций в объект без потери функционала.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

